

RideFlow — Database Lab Spring 2026

Semester Project Evaluation Rubric

Group Members: _____ Section: _____

Section A — Project Deliverables (150 Marks Total)

#	Deliverable	Requirements & Evaluation Criteria	Total Marks	Obtained Marks
D1	ERD Design	- All entities included: Users, Drivers, Vehicles, Rides, Payments, Ratings, PromoCodes, Complaints, Locations - Correct relationships defined with proper cardinality (one-to-many, many-to-many) - Primary keys, foreign keys, and attributes clearly labeled on the diagram - Diagram is clean, readable, and logically organized - Covers all 5 modules: User, Ride, Driver/Vehicle, Payment, Ratings	20	—
D2	ERD to Schema Conversion	- ERD correctly converted into a relational schema with all tables and attributes - Complete DDL statements written in MySQL for all tables - Constraints implemented: NOT NULL, UNIQUE, CHECK, DEFAULT, FOREIGN KEY - Correct and appropriate data types assigned to every column - Primary keys and foreign keys properly defined and enforce referential integrity - Many-to-many relationships correctly resolved with junction tables	30	—
D3	Complete System Implementation	- Fully functional MySQL database backend with all 5 modules implemented - All SQL queries, views, stored procedures, triggers, and events operational - Complete web or desktop UI integrated with the database - Role-based login implemented for Rider, Driver, and Admin - Live reporting and analytics dashboard functional - All database concepts (Basic SQL, Aggregates, Joins, Views, Triggers, DCL) demonstrated	100	—
DELIVERABLES TOTAL			150	— / 150

D3 - Complete System Implementation(Database Concepts & Implementation) (100 Marks Total)

#	Component	Implementation Criteria & Deliverables	Total Marks	Obtained Marks
1	Basic SQL Queries	- Correct use of SELECT, WHERE, and ORDER BY clauses - Query: list all completed rides for a specific rider ordered by date - Query: list all drivers in a city ordered by rating - Queries return accurate results and use correct table/column references	5	—
2	Aggregate Functions & HAVING Clause	- SUM() used correctly to calculate total revenue per city - AVG() used to calculate average driver ratings with HAVING AVG(score) < 3.5 filter - COUNT() used to find number of trips completed per driver - Aggregate queries grouped correctly with GROUP BY - HAVING clause filters results at group level, not row level	10	—
3	Joins for Reports	- INNER JOIN: full trip report linking Riders, Rides, Drivers, and Vehicles tables - LEFT JOIN: all riders shown including those who have never completed a ride - JOIN on Payments and PromoCodes: discount usage displayed per ride - All joins use correct ON conditions and return expected columns - Reports are readable with proper column aliasing and ordering	20	—
4	Views, Indexes & Stored Procedures	- View ActiveRidesView: shows all ongoing trips with full rider and driver details - View TopDriversView: shows only drivers with average rating above 4.5 - Indexes created on: rider_id, driver_id, ride_status, city columns - Stored procedure: auto-calculates fare using distance, duration, and surge multiplier - Stored procedure handles surge pricing logic correctly - Views and indexes are functional and queryable	15	—
5	Triggers & Events	- Trigger: automatically updates ride status to Completed when payment is marked Paid - Trigger: flags driver account and notifies admin when average rating drops below 3.5 - Trigger: increments promo code usage count when a promo is applied to a ride - MySQL Event Scheduler: expires promo codes past their expiry date every night at midnight - All triggers fire correctly under the intended conditions without errors	10	—
6	Hierarchical User Access (DCL)	- GRANT SELECT on rides table assigned to driver_role only - GRANT INSERT, SELECT on rides and payments tables assigned to rider_role - REVOKE DELETE from support_role to prevent data deletion - Admin role granted full privileges across all tables	10	—

#	Component	Implementation Criteria & Deliverables	Total Marks	Obtained Marks
		Roles are tested and access restrictions verified to work correctly		
7	User Interface	- Rider Dashboard: book a ride, view complete ride history, manage wallet balance - Driver Dashboard: toggle online/offline availability, accept or reject rides, view earnings and trip history - Admin Panel: manage users, manage vehicles, configure fare rules, view all system reports - Role-based login enforced — each dashboard is accessible only to the correct role - UI is connected to the live MySQL database (no hardcoded/mock data) - Live reporting and analytics displayed correctly in Admin Panel	30	—
IMPLEMENTATION TOTAL			100	<u> </u> / 100

Evaluated by: _____ Date: _____ Signature: _____
